

▼ Step 12 — Ridge Regression

Why Ridge?

Ridge is Linear Regression with a **penalty** added.

When we have many features, some coefficients can get very large — this causes **overfitting** (model memorizes training data but fails on new data).

Ridge **shrinks** large coefficients without removing any feature completely.

```
from sklearn.linear_model import Ridge

ridge_model = Ridge()
ridge_model.fit(X_train, y_train)
```

▼ Ridge ⓘ ?
Ridge()

```
ridge_model.score(X_train, y_train)
```

```
0.8795275304934556
```

```
ridge_y_predict = ridge_model.predict(X_test)
```

```
compare = pd.DataFrame({
    'Actual': y_test,
    'Predicted': ridge_y_predict
})
compare.head(10)
```

	Actual	Predicted	
1862	10.043249	9.626285	
1295	9.093807	9.347802	
2946	9.568015	9.492009	
2093	9.024011	9.571869	
1318	9.047821	8.741831	
502	9.230143	9.304798	
3858	11.707670	11.712720	
594	12.010666	12.567877	
903	10.425253	10.196220	
4319	8.612503	7.390702	

Next steps: [Generate code with compare](#) [New interactive sheet](#)

```
r2_score(y_test, ridge_y_predict)
```

```
0.8753267908774224
```

```
from sklearn.metrics import mean_squared_error
mse = mean_squared_error(y_test, ridge_y_predict)
rmse = np.sqrt(mse)
rmse
```

```
np.float64(0.3327916061091629)
```

Step 13 — Lasso Regression

Why Lasso?

Lasso is similar to Ridge but with one key difference:

Lasso can make some feature coefficients exactly **zero** — meaning it **removes** unimportant features automatically!

This is called **automatic feature selection** — very useful when you have many features and don't know which ones matter.

```
# Step 13 – Lasso Regression

from sklearn.linear_model import Lasso
from sklearn.metrics import r2_score, mean_squared_error

# You can tune alpha later (try 0.01, 0.1, 1, etc.)
lasso_model = Lasso(alpha=0.1)

lasso_model.fit(X_train, y_train)

# Check train score
lasso_train_score = lasso_model.score(X_train, y_train)
print("Lasso Train R²:", lasso_train_score)

# Predictions on test set
lasso_y_predict = lasso_model.predict(X_test)

# Evaluate on test set
lasso_r2 = r2_score(y_test, lasso_y_predict)
lasso_mse = mean_squared_error(y_test, lasso_y_predict)
lasso_rmse = np.sqrt(lasso_mse)

print("Lasso Test R²:", lasso_r2)
print("Lasso Test RMSE:", lasso_rmse)
```

```
Lasso Train R²: 0.8190918267596383
Lasso Test R²: 0.816100738828569
Lasso Test RMSE: 0.4041809000504551
```

Step 14 — Compare All Three Models

Why compare?

No single model is always the best. We compare R^2 and RMSE to choose the winner.

- **Higher R^2** = Better (closer to 1.0 is ideal)
- **Lower RMSE** = Better (smaller error is better)

```
# Step 14 – Compare All Three Models

models_results = {}

# Linear Regression
lin_r2 = r2_score(y_test, y_predict)
lin_mse = mean_squared_error(y_test, y_predict)
lin_rmse = np.sqrt(lin_mse)
models_results["Linear"] = {"R2": lin_r2, "RMSE": lin_rmse}

# Ridge Regression
ridge_r2 = r2_score(y_test, ridge_y_predict)
ridge_mse = mean_squared_error(y_test, ridge_y_predict)
ridge_rmse = np.sqrt(ridge_mse)
models_results["Ridge"] = {"R2": ridge_r2, "RMSE": ridge_rmse}

# Lasso Regression
models_results["Lasso"] = {"R2": lasso_r2, "RMSE": lasso_rmse}

# Show comparison as a DataFrame
results_df = pd.DataFrame(models_results).T # transpose so models are rows
results_df
```

	R2	RMSE	
Linear	0.875360	0.332747	
Ridge	0.875327	0.332792	
Lasso	0.816101	0.404181	

Next steps: [Generate code with results_df](#) [New interactive sheet](#)

Step 15 — Actual vs Predicted Plot

Why?

Numbers alone don't tell the full story. This plot shows visually how close our predictions are to the actual prices.

- If points lie **on the red diagonal line** → perfect predictions
- Points **scattered away** from the line → prediction errors

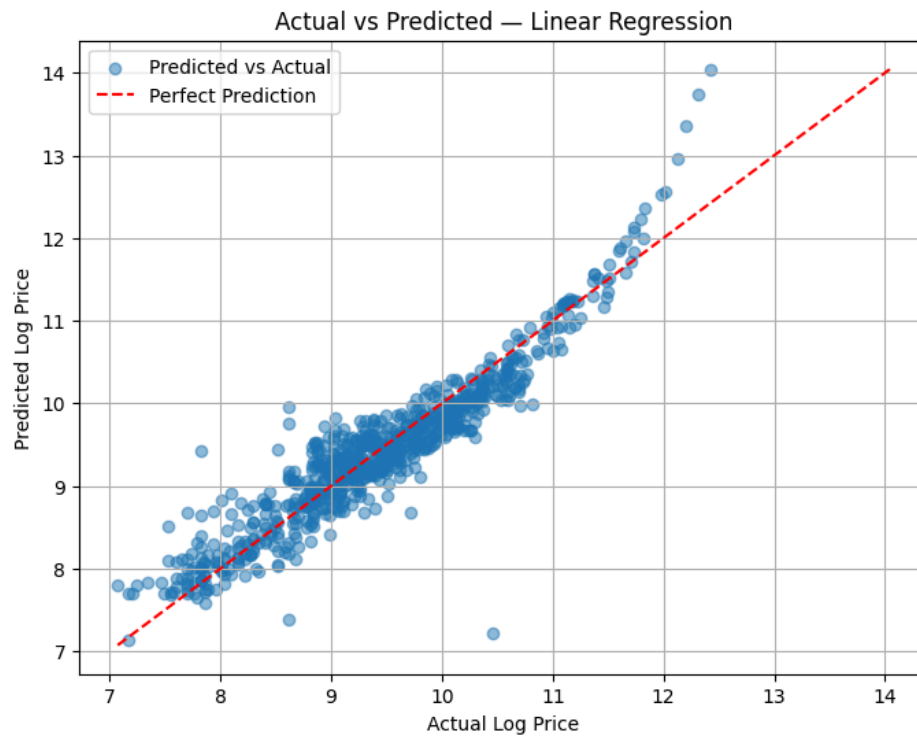
```
# Step 15 – Actual vs Predicted Plot

plt.figure(figsize=(8, 6))

# Scatter plot of Actual vs Predicted
plt.scatter(y_test, y_predict, alpha=0.5, label="Predicted vs Actual")

# Perfect prediction line
min_val = min(y_test.min(), y_predict.min())
max_val = max(y_test.max(), y_predict.max())
plt.plot([min_val, max_val],
         [min_val, max_val],
         color="red",
         linestyle="--",
         label="Perfect Prediction")

plt.xlabel("Actual Log Price")
plt.ylabel("Predicted Log Price")
plt.title("Actual vs Predicted – Linear Regression")
plt.legend()
plt.grid(True)
plt.show()
```



```
plt.figure(figsize=(8, 6))

# Scatter plot of Actual vs Predicted
plt.scatter(y_test, ridge_y_predict, alpha=0.5, label="Predicted vs Actual")

# Perfect prediction line
min_val = min(y_test.min(), y_predict.min())
max_val = max(y_test.max(), y_predict.max())
plt.plot([min_val, max_val],
         [min_val, max_val],
         color="black",
         linestyle=":",
         label="Perfect Prediction")

plt.xlabel("Actual Log Price")
plt.ylabel("Predicted Log Price")
plt.title("Actual vs Predicted — Ridge Regression")
plt.legend()
plt.grid(True)
plt.show()
```

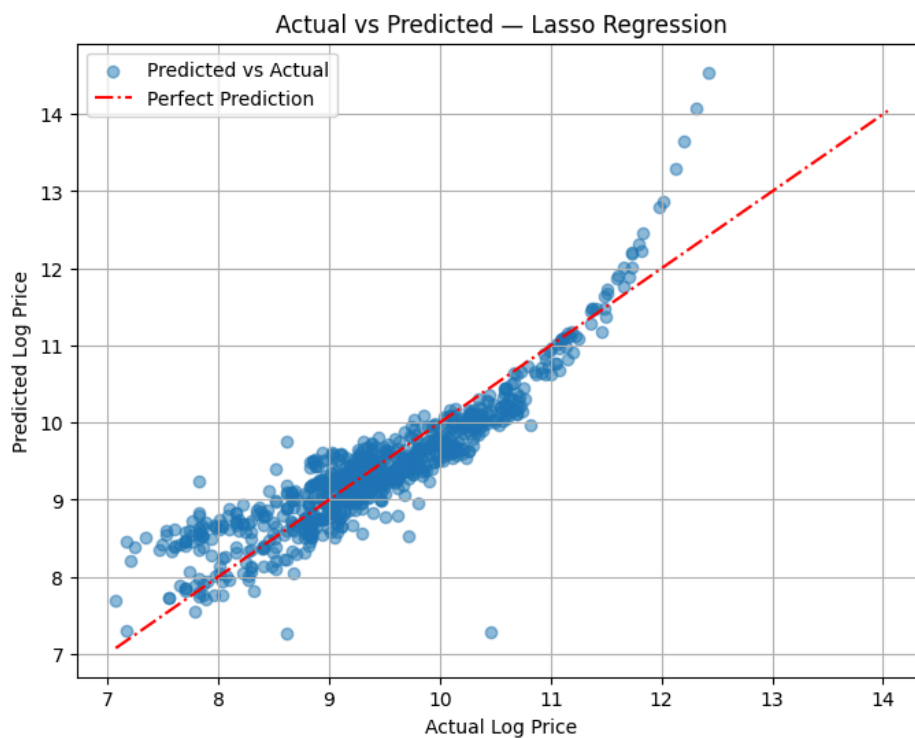
Actual vs Predicted — Ridge Regression

```
plt.figure(figsize=(8, 6))

# Scatter plot of Actual vs Predicted
plt.scatter(y_test, lasso_y_predict, alpha=0.5, label="Predicted vs Actual")

# Perfect prediction line
min_val = min(y_test.min(), y_predict.min())
max_val = max(y_test.max(), y_predict.max())
plt.plot([min_val, max_val],
         [min_val, max_val],
         color="red",
         linestyle="-.",
         label="Perfect Prediction")

plt.xlabel("Actual Log Price")
plt.ylabel("Predicted Log Price")
plt.title("Actual vs Predicted — Lasso Regression")
plt.legend()
plt.grid(True)
plt.show()
```



Start coding or [generate](#) with AI.